

CADENCE: Case-Grounded, Deliberation-Driven, and Solver-Verified Architectural Decision-Making with Large Language Models

Quang-Vinh Dang

Abstract—Large language models (LLMs) are increasingly proposed as assistants for architectural decision-making in service-oriented systems, yet most existing approaches either generate an architecture decision record (ADR) from a single unconstrained LLM call, retrieve similar precedents without reconciling the competing quality-attribute concerns a real decision must balance, or debate among multiple agents without any formal check that the resulting decision is realizable within a bounded budget of architectural tactics. We present CADENCE, a four-stage algorithm combining (1) retrieval-augmented case-based reasoning over a corpus of real, mined ADRs, (2) knowledge-graph-grounded multi-agent deliberation in which each agent advocates for one ISO/IEC 25010-derived quality attribute, (3) constraint-solver-verified feasibility, encoded as a two-phase weighted MaxSAT/SMT problem in Z3 and repaired via an LLM-driven loop targeting the solver’s uncovered-attribute signal (its unsatisfiable core is reported only as a supplementary diagnostic), and (4) a separate self-critique pass blending a deterministic structural utility score with a qualitative LLM judgment. Unlike prior multi-agent architectural decision systems, whose agents occupy sequential functional roles and whose “evaluation” step is itself another LLM opinion, CADENCE’s agents represent adversarial quality-attribute concerns and its feasibility check is a formal solver, not a second language model. We implement the full pipeline against a real corpus of 6,173 mined ADRs and evaluate it on two independently-sampled held-out sets ($N = 3$ each, real end-to-end local-model wall-clock cost being the binding constraint on sample size) against four baselines forming an ablation ladder — zero-shot generation, retrieval-only generation, multi-agent deliberation without solver verification, and CADENCE without self-critique — on BERTScore, BLEU, ROUGE-1, and METEOR, plus two metrics only a solver-verified system can report: constraint-satisfaction rate and repair-loop convergence. At the one tested budget where feasibility is arithmetically possible ($B = 5$), constraint satisfaction is 0% and repair never converges even with its full design budget — an honest negative result we diagnose mechanistically rather than soften; the tighter budgets we also test are infeasible by construction and reported only as construction checks. Against this, self-critique shows a modest but consistent positive signal: CADENCE with self-critique scores higher than without it in 7 of 8 system-budget-metric comparisons, the only ablation where adding a stage helps rather than being flat. We separately find surface n-gram metrics penalize CADENCE’s deliberately terse, solver-parseable output relative to baselines prompted for open-ended prose, while BERTScore remains comparable across all systems; we argue this is an asymmetry, not a quality gap,

with implications for evaluating structured multi-stage LLM pipelines generally. Beyond our own re-implemented baselines, we locate our exact held-out items within a public replication package’s real, published generations from four frontier models and compare directly against them, giving an external reference point our baselines alone cannot provide.

Index Terms—Large language models, architectural decision-making, architecture decision records, multi-agent systems, constraint solving, retrieval-augmented generation, self-critique, service-oriented architecture.

I. INTRODUCTION

SERVICE-ORIENTED systems accumulate architectural decisions continuously: how to scale a read-heavy service, whether to introduce a message queue, where to draw a service boundary, how to trade encryption overhead against latency. Each such decision commits a team to consequences across multiple, frequently conflicting quality attributes — performance against maintainability, scalability against cost, security against usability — and each is, in practice, made under time pressure, with incomplete visibility into precedent, and without a systematic check that the chosen tactics are mutually consistent within what the team can actually staff and afford. Architecture decision records (ADRs) exist to make this reasoning explicit and reviewable, yet writing one well — surveying precedent, weighing competing quality attributes on their merits, and verifying that the resulting commitments do not silently exceed what the team can deliver — is exactly the kind of effortful reasoning that teams under delivery pressure are most likely to shortcut.

Large language models are a natural candidate to assist here, and a growing body of work explores LLM-assisted architectural decision-making and ADR generation. The most direct approaches prompt a single LLM, zero-shot or with light context, to draft a decision from stated requirements [1]. A second line grounds generation in retrieval over real, mined ADR corpora, anchoring output in precedent rather than parametric knowledge alone [2]. A third introduces multiple LLM agents, most notably a four-agent pipeline of functional roles — an Analyst, a Modeler, a Designer, and an Evaluator — each handling a different phase of the decision [4]. Each line solves part of the problem this paper addresses, but none solves all of it, and none can answer a question that matters for any decision with real operational consequences:

Q.-V. Dang is with British University Vietnam, Hung Yen, Vietnam (e-mail: vinh.dq4@buv.edu.vn).

Manuscript submitted to the IEEE Transactions on Services Computing Special Issue on “Large Language Models in Service-Oriented Ecosystems Design: Advances and Applications,” 2026.

is the decision the model just wrote actually feasible, given a bounded, explicit budget of architectural commitments the team can realistically execute? A single LLM call has no mechanism to check this. Retrieval grounds the decision in precedent but does not reconcile competing quality-attribute demands, let alone verify them formally. And a multi-agent pipeline whose final “Evaluator” step is itself another LLM call is still, ultimately, one model’s opinion of another’s output — persuasive, perhaps, but not a proof of anything.

This paper presents CADENCE, a four-stage algorithm designed around this gap directly. CADENCE (1) retrieves precedent decisions from a real corpus of mined, human-authored ADRs via dense vector search, so deliberation starts from what teams have actually decided before, not a blank context window; (2) stages a bounded, structured multi-agent deliberation in which each agent is anchored to one ISO/IEC 25010-derived [9] quality attribute — performance, security, maintainability, scalability, and cost/operability — and argues for the candidate decision that best serves its attribute, grounded in a knowledge graph of architectural tactics, so the competing concerns a real decision must balance are represented explicitly and adversarially rather than folded into one model’s undifferentiated judgment, with the graph’s cross-attribute trade-off structure carried forward into Stage 3’s feasibility check and Stage 4’s utility scoring; (3) encodes the deliberated candidate’s implied tactic commitments as a two-phase weighted MaxSAT/SMT problem checked with the Z3 theorem prover, and, if infeasible under a caller-specified tactic budget, constructs a targeted repair prompt from the solver’s uncovered-attribute signal (its unsatisfiable core, when available, is a supplementary diagnostic, not the repair driver) and issues it as a standalone repair call, iterating to a fixed cap and degrading gracefully with an explicit caveat if feasibility is never reached; and (4) finalizes the solver-verified decision through a separate self-critique pass combining a deterministic structural utility score — derived from which quality attributes the selected tactics actually cover, and what trade-offs they incur — with a qualitative LLM judgment of remaining weaknesses, so the system’s confidence is never purely self-reported by the same reasoning process that produced the decision. Each stage closes a specific failure mode the others leave open: retrieval alone cannot reconcile competing quality attributes; deliberation alone cannot verify feasibility; and an unverified, unaudited decision cannot be trusted as final without an independent finalization pass.

We implement the complete CADENCE pipeline end-to-end against a real, publicly available corpus of 6,173 mined ADRs drawn from 882 open-source repositories [6], and evaluate it against four baselines forming an ablation ladder, each adding exactly one stage over the previous: zero-shot generation, retrieval-only generation, multi-agent deliberation without solver verification, and CADENCE without self-critique (adds Stage 3’s solver-verified feasibility and repair loop, but not Stage 4). This ladder isolates retrieval’s, deliberation’s, solver verification’s, and self-critique’s contributions individually. We report the standard generation-quality metrics used in prior ADR-generation work — BERTScore, BLEU, ROUGE-1, and METEOR — against held-out, human-authored ground truth,

alongside two metrics only a solver-verified system can report: constraint-satisfaction rate and repair-loop convergence. Our results surface a methodologically important asymmetry: surface n-gram overlap metrics (ROUGE-1, METEOR) penalize CADENCE’s deliberately terse, structured output — terse by design, so Stages 3 and 4 can parse it reliably — relative to baselines prompted for open-ended prose closer in length to the reference ADRs, while BERTScore, far less sensitive to this length mismatch, remains comparable across all four systems. We report this honestly rather than letting a metric artifact of our own prompt design misrepresent the comparison, and discuss what it implies for evaluating structured, multi-stage LLM pipelines whose intermediate representations are deliberately not free-form prose.

This work makes three contributions. First, CADENCE, to our knowledge the first architectural decision-making algorithm combining real-corpus retrieval, adversarial quality-attribute-grounded multi-agent deliberation, formal constraint-solver verification with a solver-in-the-loop repair mechanism, and a separately-scored self-critique finalization stage. Second, a concrete construction for turning free-text, LLM-authored deliberation into a solvable constraint-satisfaction instance, with one design choice empirically validated during development: a lexicographic, rather than weighted-sum, multi-objective optimization is required to make coverage strictly dominate trade-off avoidance regardless of caller-supplied weight magnitudes, a subtlety a naive encoding gets wrong (Section III-D). Whether the full construction reliably reaches feasibility in practice is an open question our small-sample evaluation (Section IV) does not yet settle — we report a fully-diagnosed negative result rather than a positive one. Third, a real, if small-sample, empirical evaluation against real baselines on a real corpus — including a direct comparison, on identical held-out items, against real published generations from four frontier models via an independent public replication package (Section IV-E) — surfacing a metric asymmetry (BERTScore versus surface n-gram overlap) we believe generalizes to other structured, multi-stage LLM systems.

The remainder of this paper is organized as follows. Section II positions CADENCE against the closest prior systems for LLM-assisted architectural decision-making and against the broader techniques it draws on. Section III presents the CADENCE algorithm in full, stage by stage. Section IV describes our experimental setup, baselines, metrics, and results. Section V discusses limitations, threats to validity, and the metric-asymmetry finding. Section VI concludes.

II. RELATED WORK

A. LLM-Assisted Architectural Decision-Making

Dhar et al. [1] demonstrate that a single LLM, prompted with a stated architectural problem, can produce a plausible ADR, but their approach neither retrieves precedent nor verifies the result against any formal notion of feasibility: the model’s output is accepted as-is. Follow-on work from the same group frames ADR authoring as an explicit drafting process rather than one-shot generation [7], and a separate recent study evaluates LLMs specifically as *detectors* of violations against already-recorded architectural decisions [8] —

a complementary, downstream concern rather than assisting the decision itself. A complementary line of work treats the growing body of real, mined ADRs as a resource in its own right: large-scale mining studies of ADR adoption in open-source projects [6] establish both the raw corpus this paper’s retrieval stage draws on and the empirical observation — corroborated in our own corpus statistics — that ADR adoption per repository is sparse, which any retrieval-based system must tolerate rather than assume away. Building on such corpora, retrieval-grounded generation work [2], [3] shows that conditioning an LLM’s ADR generation on retrieved precedent measurably improves output quality, and explicitly frames the task this paper’s held-out evaluation also uses: given only a decision’s title, generate the body. This precedent motivates our own held-out query construction (Section IV) and supplies both the retrieval-only baseline and the corpus itself. Neither line of work, however, reconciles multiple quality-attribute concerns or checks feasibility formally — gaps CADENCE is designed to close.

The closest prior system to CADENCE is MAAD [4], a four-agent pipeline in which an Analyst, a Modeler, a Designer, and an Evaluator agent each handle one sequential phase of the decision process; a related paper from an overlapping author group targets generating the *rationale* behind an already-made decision rather than the decision itself [5]. MAAD and CADENCE share the premise that a single LLM call is insufficient and multiple agents should participate, but the two systems differ in three load-bearing ways. First, MAAD’s agents occupy sequential *functional* roles — each does a different *kind* of work at a different pipeline point — whereas CADENCE’s agents occupy simultaneous, *adversarial* roles: every agent does the same kind of work (propose and critique a candidate) but from a different, often conflicting quality attribute, so the tension between (e.g.) a performance-motivated tactic and its maintainability cost is an argument between agents rather than resolved silently inside one agent’s reasoning. Second, MAAD’s Evaluator is itself an LLM judge assessing the Designer’s output with the same kind of model that produced it, so MAAD’s final confidence is ultimately one language model’s opinion of another’s work. CADENCE’s Stage 3 feasibility check is a formal constraint solver over an explicit tactic budget and coverage requirement; a decision either satisfies the encoded constraints or not, independent of any model’s subjective assessment, and when it does not, the solver’s uncovered-attribute signal gives the repair loop a structured target rather than generic revision instructions (the unsatisfiable core is also reported, as a supplementary diagnostic). Third, CADENCE’s retrieval stage is grounded in a large, real, citable corpus of mined ADRs, giving deliberation a case-based starting point that MAAD’s requirements-only formulation does not provide. We discuss MAAD further, and compare against a solver-free multi-agent ablation modeled directly on this difference, in Section IV. Table I summarizes this positioning.

B. LLMs for Service-Oriented and Microservice Architecture

A recent review of LLMs across the service-computing lifecycle names architectural decision support an open direction,

not a solved one [11] — the two concrete service-oriented design systems that exist target a different lifecycle point: one recommends monolith-to-microservice decompositions via contrastive learning [12], the other synthesizes candidate microservice architectures from textual requirements [13]. Both decompose or synthesize a service *topology*; neither decides and justifies one architectural *commitment* against competing quality attributes the way CADENCE does, so the gap this review names is still open for this special issue’s theme specifically.

C. LLM–Solver Integration

Combining LLMs with formal solvers to check what a model proposes is an active area outside the architecture domain: representative approaches translate natural-language problems into a formal logical representation and dispatch it to a symbolic solver so the solver, not the LLM, handles the deductive step [14], and closely related work verifies LLM-authored natural-language proofs by compiling them into a checkable formal representation [15]. More recent work bridges LLMs and symbolic solvers directly through the Model Context Protocol, giving a model a structured tool interface to a solver rather than a one-shot translation [16], and, closest in spirit to CADENCE’s Stage 3, one planning system drives an LLM repair loop from a Z3 solver’s unsatisfiable core when a proposed plan is infeasible [17] — the same solver-in-the-loop repair pattern applied here to architectural tactic feasibility rather than task planning. These techniques target logical inference, formal proof checking, and classical planning; CADENCE applies the same pattern — let the LLM produce a candidate, let a solver check it, feed the failure signal back for repair — to a softer target: whether a decision’s implied tactic commitments are jointly satisfiable within an explicit budget and coverage requirement, encoded as weighted MaxSAT/SMT rather than propositional or first-order logic or a planning domain. To our knowledge, no prior work applies solver-in-the-loop verification to architectural trade-off feasibility specifically; we cite this literature as prior art for the LLM-plus-solver *technique*, not the specific *application*.

D. Multi-Agent Deliberation and Self-Critique with LLMs

Multi-agent debate among LLM instances has been shown to improve factual accuracy and reasoning quality over single-model generation by exposing a claim to independent critique before acceptance [18], an effect corroborated by independent work showing debate encourages divergent thinking a single model’s self-reflection alone does not reliably produce [19], and extended to using multi-agent debate itself as an evaluation mechanism [20]. CADENCE’s Stage 2 applies this general finding to a domain-specific structure: rather than generic debate, each agent’s role and critique perspective is fixed to one ISO/IEC 25010 quality attribute and grounded in a shared knowledge graph of architectural tactics, anchoring debate to a concrete, checkable vocabulary, with the graph’s cross-attribute trade-offs consumed downstream by Stage 3’s feasibility check and Stage 4’s utility scoring rather than

TABLE I
POSITIONING AGAINST THE CLOSEST PRIOR LLM-ASSISTED ARCHITECTURAL DECISION-MAKING SYSTEMS

System	Real-corpus retrieval	Multi-agent	Agent roles	Formal solver verification	Separate critique stage
Dhar et al. [1]	No	No	—	No	No
Context Matters [2]	Yes	No	—	No	No
MAAD [4]	No	Yes (4 agents)	Sequential functional roles (Analyst / Modeler / Designer / Evaluator)	No (LLM judge)	No (folded into pipeline)
CADENCE (this work)	Yes	Yes	Simultaneous adversarial quality-attribute advocates	Yes (Z3 weighted MaxSAT/SMT + repair loop)	Yes (distinct utility-scoring pass)

argued over within Stage 2 itself. Separately, self-refinement work shows a model can improve its own output by critiquing and revising it without external supervision [21], and verbal-reinforcement approaches extend this using episodic memory of past failures [22]. This literature is not uniformly optimistic: a widely-cited negative result finds LLMs prompted to self-correct *without external feedback* frequently fail to improve, and can even degrade an already-correct answer [23] — a finding we address directly in Stage 4’s design (Section III-E) and threats-to-validity discussion (Section V). CADENCE’s Stage 4 draws on the self-refinement idea but departs from both strands in one respect important for high-stakes decisions: self-critique is not a further revision of the same reasoning chain in isolation, but an independently-scored finalization combining a deterministic structural component — computed from Stage 3’s solver output, not re-derived by an LLM — with the qualitative LLM judgment, so final confidence is never solely the same model marking its own homework without external grounding.

E. Case-Based Reasoning and Retrieval-Augmented Generation

Case-based reasoning — solving new problems by adapting solutions to similar past ones — is a natural fit for architectural decision-making, since real decisions are rarely made from first principles but from precedent and prior experience; recent work formalizes how its classical case-retrieve-reuse-revise cycle maps onto LLM agents [27], and a concrete instantiation combines case-based retrieval with generation for a different high-stakes domain (legal question answering) [26]. CADENCE’s Stage 1 operationalizes this idea with dense vector retrieval over a real ADR corpus rather than a hand-curated case base, following the same retrieval-augmented generation methodology increasingly used to ground LLM generation in verifiable evidence [24], including in software engineering, where a recent survey catalogs repository-level retrieval strategies for code generation paralleling this paper’s corpus-level retrieval for architectural precedent [25]. The architectural tactics structuring CADENCE’s knowledge graph and constraint encoding — caching, connection pooling, horizontal scaling, circuit breakers, and the like, with their well-known cross-attribute trade-offs — are an original synthesis informed by and in the spirit of the established software architecture tactics catalog [10], not a reproduction of its text, operationalized by this paper’s Stage 2/3 machinery into a form an LLM-driven deliberation and a formal solver can both act on.

III. THE CADENCE ALGORITHM

A. Problem Formulation and Overview

CADENCE takes as input a natural-language decision context c (the requirements and situational description a real ADR’s “Context” section would state) and a set of required quality attributes $Q = \{q_1, \dots, q_5\} \subseteq \{\text{performance, security, maintainability, scalability, cost_operability}\}$, the ISO/IEC 25010-derived taxonomy fixed for this work. It produces a finalized, provenance-complete ADR: a decision statement, a rationale, a per-quality-attribute utility score, a feasibility verdict with selected tactics and residual gaps, and a full transcript of the precedents consulted, agent positions debated, and repair iterations taken. The four stages — retrieval, deliberation, solver verification with repair, and self-critique finalization — execute in sequence, each consuming the previous stage’s output and addressing a distinct failure mode the others do not cover. Figure 1 shows the overall flow, and Algorithm 1 gives it formally.

B. Stage 1: Retrieval-Augmented Case-Based Reasoning

Stage 1 embeds the decision context c with a sentence-embedding model and retrieves the k nearest precedent ADRs, by cosine similarity, from a dense vector index built over the full corpus of 6,173 real, mined ADRs (Section IV). Each corpus record is parsed leniently from its source Markdown file — real-world ADR filenames and internal structure are inconsistent across repositories, so the parser extracts only a title and the full raw body text rather than assuming a fixed section schema. Retrieval returns the top- k precedent records, whose titles are passed into Stage 2 as grounding context; when used inside a held-out evaluation, the query record’s own ground-truth ADR is excluded from its own retrieval results to prevent leakage.

C. Stage 2: Knowledge-Graph-Grounded Multi-Agent Deliberation

Stage 2 instantiates one `QualityAttributeAgent` per required quality attribute in Q . Each agent is grounded in a knowledge graph G built from a hand-curated catalog of 26 architectural tactics spanning the five quality attributes (5 or 6 per attribute; Table II lists the full catalog), where each tactic node has a directed *supports* edge to the quality attribute it primarily serves and directed *trade-off* edges, annotated with a natural-language explanation, to every other attribute it is known to burden — for instance, *caching*

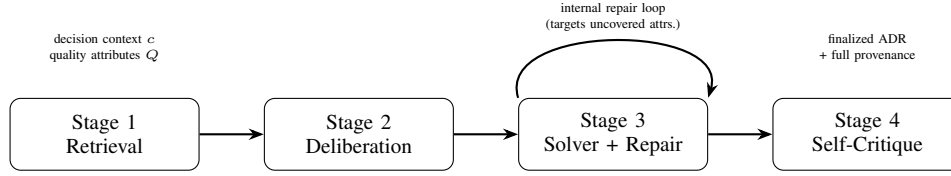


Fig. 1. The CADENCE pipeline. Each stage consumes the previous stage’s output; Stage 3’s repair loop is internal to Stage 3 — it never re-enters Stage 2’s multi-agent deliberation — and issues a standalone repair call driven by the solver’s uncovered-attribute signal (with the unsatisfiable core reported only as a supplementary diagnostic), iterating to a fixed cap before Stage 4 finalizes the decision.

Algorithm 1 CADENCE

Require: context c , quality attributes Q , tactic budget B , tactic catalog with graph G , max deliberation rounds R , max repair iterations K

Ensure: finalized decision with provenance

```

1:  $P \leftarrow \text{RETRIEVE}(c)$  {Stage 1: top- $k$  precedents}
2:  $(d, r) \leftarrow \text{DELIBERATE}(c, P, Q, G, R)$  {Stage 2: bounded multi-agent debate}
3:  $i \leftarrow 0$ ;  $best \leftarrow \perp$ 
4: loop
5:    $res \leftarrow \text{CHECKFEASIBILITY}(d, Q, B, G)$  {Stage 3, Phase 1+2}
6:   if  $best = \perp$  or  $|res.covered| > |best.res.covered|$  then
7:      $best \leftarrow (res, d, r)$  {track highest-coverage attempt seen so far}
8:   end if
9:   if  $res.feasible$  or  $i = K$  then
10:    break {degrade gracefully with a caveat if infeasible}
11:  end if
12:   $(d, r) \leftarrow \text{REPAIR}(d, res.uncovered, B)$  {standalone LLM repair call, targets uncovered attributes}
13:   $i \leftarrow i + 1$ 
14: end loop
15:  $(res, d, r) \leftarrow best$  {use the best attempt seen, not necessarily the last}
16:  $u \leftarrow \text{STRUCTURALUTILITY}(res, G)$  {Stage 4, deterministic}
17:  $q \leftarrow \text{QUALITATIVECRITIQUE}(d, r, Q)$  {Stage 4, separate LLM pass}
18: return  $\text{FINALIZE}(d, r, u, q, res, P)$ 
  
```

supports *performance* but trades off against *maintainability* via cache-invalidation complexity, and *authentication* supports *security* but trades off against both *performance* (verification latency) and *cost_operability* (identity-infrastructure upkeep), as Figure 2 depicts. Given the decision context and Stage 1’s retrieved precedents, each agent proposes a candidate decision serving its own attribute using G ’s *supports* edges, and may critique candidates proposed by other agents; G ’s *trade-off* edges are consumed downstream by Stage 3’s feasibility optimization and Stage 4’s utility score, rather than injected into Stage 2’s prompts. Deliberation proceeds for a bounded number of rounds (`max_rounds`), after which an orchestrator synthesizes the debate into one converged candidate decision and rationale.

Synthesis is the stage’s most format-fragile step:

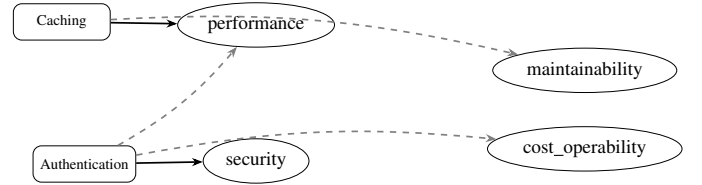


Fig. 2. Excerpt of knowledge graph G : solid arrows are *supports* edges, dashed gray arrows are *trade-off* edges. Only 2 of the catalog’s 26 tactics and 4 of its 5 quality attributes are shown for legibility.

it asks the underlying LLM to emit a structured CANDIDATE/RATIONALE response that Stage 3 can parse deterministically. In practice, real small-model output deviates from any single expected format in ways a fixed parser cannot fully anticipate in advance (Section III-F); the orchestrator therefore retries the same synthesis prompt up to three times on a parse failure before surfacing an error, since generation is stochastic and a differently-sampled retry frequently self-corrects a formatting slip that a hand-written regex extension would not have anticipated.

D. Stage 3: Constraint-Solver-Verified Feasibility with Repair Loop

Stage 2’s converged candidate is free text: a human-readable decision and rationale, not a structured commitment a solver can reason about directly. Stage 3 first extracts which of the 26 catalog tactics the candidate text actually mentions, via fuzzy word-stem overlap against each tactic’s name, giving Boolean decision variables $\{t_1, \dots, t_m\}$, one per mentioned tactic. It then checks feasibility in two phases against the Z3 SMT solver.

Phase 1 (strict feasibility). A budget constraint $\sum_i t_i \leq B$ (a caller-specified tactic budget B) and, for every required quality attribute $q \in Q$, a coverage constraint $\bigvee_{i: \text{supports}(t_i)=q} t_i$ (at least one selected tactic must support q) are asserted with `assert_and_track`, so that if the instance is unsatisfiable, Z3 returns an *unsatisfiable core*: the constraints jointly responsible for infeasibility. This core is a useful, but not necessarily complete, diagnostic — Z3’s core-extraction semantics can name only one of several symmetric zero-coverage attributes as blocking, omitting others equally unsatisfied — so Stage 3 does not treat it as authoritative.

Phase 2 (best-effort optimized selection). A separate Optimize instance re-asserts the budget constraint and searches for the tactic selection that jointly (a) maximizes how many required quality attributes are covered and (b) minimizes the caller-weighted cost of the trade-offs incurred, always returning a concrete selection even when Phase 1

TABLE II
THE 26-TACTIC CATALOG GROUNDING STAGE 2’S KNOWLEDGE GRAPH G

QA	Tactic	Primary Effect	#TO	QA	Tactic	Primary Effect	#TO
Perf.	Caching	Avoid recomputation/I-O	1	Maint.	Automated regression tests	Catch regressions pre-production	1
	Connection pooling	Reuse expensive connections	1		Centralized structured logging	Make behavior inspectable	2
	Async. msg. queues	Decouple slow work from request path	2		Dependency injection / IoC	Decouple concrete dependencies	1
Sec.	Data denormalization	Avoid expensive joins/calls	1	Scal.	Horizontal scaling	Add instances, not grow one	2
	Read-through/write-behind	Batch/defer writes, prefetch	2		Database sharding	Partition data by shard key	2
	Authentication	Verify caller identity	1		Event-driven architecture	Events, not sync calls	2
	Fine-grained authz.	Restrict callers’ permissions	1		Auto-scaling infrastructure	Add/remove capacity per load	1
	Encryption (transit/rest)	Protect intercepted/disk data	2	Cost	Read replicas	Serve reads from replicas	1
	Rate limiting/throttling	Cap per-caller load	1		Managed cloud services	Vendor-operated, not self-hosted	2
	Least-privilege boundaries	Isolate blast radius	1		Infrastructure as code	Declarative reproducibility	0
Maint.	Input validation	Reject malformed input early	1		Multi-tenancy	Share infra across tenants	2
	Modular decomposition	Split along business boundaries	2		Serverless / FaaS	Pay per invocation	2
	Versioned API contracts	Evolve without breaking consumers	1		Circuit breaker	Stop calling failing dependency	1

#TO: number of *trade-off* edges the tactic carries to other quality attributes in G (0–2 in this catalog); QA: the quality attribute the tactic’s *supports* edge targets. Full trade-off targets and natural-language rationale for each edge are in `src/deliberation/knowledge_graph.py` (see Data and Code Availability).

found the instance strictly infeasible. These two objectives are declared under Z3’s `priority="lex"` lexicographic mode as separate `add_soft` groups, rather than one weighted-sum objective — a distinction we verified matters empirically. An earlier weighted-sum formulation used a single large constant to bias coverage above trade-off cost, but because trade-off weights are caller-supplied with no upper bound, a selection accumulating enough trade-off weight could mathematically outvote a required attribute’s coverage regardless of the bias constant’s size (a concrete repro during development: four trade-offs at weight 300 each outweighed a coverage bias of 1000). Lexicographic priority makes coverage strictly dominate trade-off avoidance independent of weight magnitude — the only encoding safe for caller-supplied weights whose scale cannot be bounded in advance. The resulting `covered_quality_attributes` / `uncovered_quality_attributes` split from this phase, not Phase 1’s `unsat` core, is the sound and complete signal of what still needs fixing.

If Phase 1 finds the instance infeasible, Stage 3 constructs a repair prompt naming, for every uncovered attribute, the concrete tactics that would cover it, and asks the LLM to revise the candidate within the same budget. This is re-checked against the solver, iterating up to `max_repair_iterations` times and tracking the best (highest-coverage) attempt seen. If the budget is never satisfiable within the cap, Stage 3 returns the best attempt found with an explicit caveat rather than silently accepting an infeasible decision or discarding an otherwise-reasonable candidate; a transient LLM or parse failure during any repair attempt triggers the next iteration, not an abort.

E. Stage 4: Self-Critique Finalization

Stage 4 combines two independently-computed signals into each quality attribute’s final utility score. The *structural* component is deterministic: for attribute q , it is 1.0 if q is among the solver-verified decision’s covered attributes, minus a fixed penalty of 0.2 for every incoming trade-off edge from a selected tactic onto q in G , floored at 0.0; it depends only on Stage 3’s already-verified output and involves no further LLM call. The *qualitative* component is a single LLM pass scoring every required attribute on a 0–10 scale and flagging

any residual weakness, given the full decision, rationale, and attribute list in one call; parsing uses a three-tier matcher — a strict same-line pattern, a tolerant same-line fallback, and a heading-section fallback for responses that state an attribute as its own heading with the field on a separate, unlabeled line (Section III-F) — so a loose regex cannot lock onto a prose preamble instead of the real scored line, and the call is retried up to three times on a parse failure as Stage 2’s synthesis call is. The two components combine as $\text{combined}_q = 0.5 \cdot \text{structural}_q \cdot 10 + 0.5 \cdot \text{qualitative}_q$, and overall decision utility is the mean combined score across all required attributes. The finalized `FinalADR` carries this scoring together with full provenance — retrieved precedents, the complete per-agent deliberation transcript, which tactics were selected and by which repair iteration, and any solver caveat — so the decision is auditable end-to-end.

F. Implementation Robustness: Parsing Real, Stochastic LLM Output

A structured-output prompt’s hand-written happy-path parser reliably breaks against a real, deployed small language model, and our development process surfaced this repeatedly and concretely, not as a theoretical concern. Running the full pipeline end-to-end against Qwen2.5-1.5B-Instruct, the reproducible local backbone used throughout this work, produced six distinct real formatting deviations across Stages 2–4, summarized in Table III.

The first four are addressable by a sufficiently tolerant same-line parser. The fifth is not, and its discovery motivated this work’s general strategy of pairing every structured-output prompt with a bounded retry (two to three attempts) of the identical prompt on a parse failure, rather than special-casing wording variants indefinitely; because generation is stochastic, a differently-sampled retry frequently self-corrects a slip the parser did not anticipate, and Stages 2, 3, and 4 all apply this pattern. The sixth, discovered later during the scaled evaluation run (e.g. `**Performance**` followed, several lines later, by `**Score:** 9/10` with no attribute name on that line), shows retry’s limit: it exhausted all three attempts because it was not a stochastic one-off but a genuinely different *structure* no same-line pattern, however tolerant, could match — fixing

TABLE III
SIX REAL LLM OUTPUT-FORMAT DEVIATIONS ENCOUNTERED

#	Deviation	Resolved by
1	Extra descriptor word (“Candidate Decision:” for “Candidate:”)	Tolerant same-line parser
2	Non-literal “no weakness” (“None identified” for literal “none”)	Tolerant same-line parser
3	Fraction-style score (“8/10”), this model’s <i>default</i> 0–10 answer	Tolerant same-line parser
4	Prose preamble a loose regex could lock onto instead of the real line	Tolerant same-line parser
5	Genuine word substitution (“Candidacy:” for “Candidate:”), unanticipatable by regex tolerance	Bounded retry (2–3 attempts)
6	Attribute-as-heading structure: attribute on its own heading line, field unlabeled on a later line	Third parser tier (heading-section fallback)

it required a third parser tier (Section III-E) that locates an attribute’s heading and searches only the bounded section up to the next heading for an unlabeled field. We report this as a methodological finding in its own right: any system asking an LLM for structured intermediate output a downstream deterministic component must consume should validate its parser against real, repeated model output before trusting it unattended, treat bounded retry as the general-purpose fix for wording variation, and expect that a genuinely new structural format will eventually require a genuinely new parser tier — discoverable only by actually running the real pipeline.

G. Worked Example

To make the four stages concrete, this section walks through one real, complete run of the pipeline reported verbatim from the actual implementation (`scripts/run_worked_example.py`) — illustrative, not an evaluation data point; this paper’s quantitative claims come from Section IV’s held-out runs, not this example.

Given a context describing a 10x read-traffic surge on an order-processing service, with requirements to keep p99 latency low, stay operable by a small team, and avoid new security risk, **Stage 1** retrieves three real precedents on message brokering, server architecture, and JWT inter-service authentication. **Stage 2** runs two rounds across the five agents: the *performance* agent proposes a message queue to decouple reads from the application server; the *security* agent independently argues for JWT authentication, encryption, and rate limiting; the *cost_operability* agent’s round-2 critique flags a real cross-attribute tension, that the message broker’s added network overhead “could affect real-time interactions.” The orchestrator synthesizes these into one converged candidate, from which Stage 3 extracts a tactic selection.

Stage 3 extracts three catalog tactics — *asynchronous processing via message queues*, *event-driven architecture*, and *serverless/FaaS deployment* — covering *performance*, *scalability*, and *cost_operability*, but not *maintainability* or *security*. This example runs at $B = 4$, which, as Section IV establishes, cannot cover all five attributes under a catalog where each tactic supports exactly one — unlike the scaled run’s $B = 5$ condition, it is not a test of achievable-budget feasibility. It nonetheless illustrates the repair loop’s mechanics: one tactic slot remained within budget, but the candidate’s text never

TABLE IV
WORKED EXAMPLE (ILLUSTRATIVE, $B = 4$, NOT AN EVALUATION DATA POINT): PER-ATTRIBUTE UTILITY SCORES (OVERALL: 6.6/10)

Quality Attribute	Structural	Qualitative	Combined
Performance	0.8	8.0	8.0
Security	0.0	9.0	4.5
Maintainability	0.0	7.0	3.5
Scalability	1.0	8.0	9.0
Cost/Operability	1.0	6.0	8.0

named a fourth, security- or maintainability-covering tactic for the extractor to find. Both repair attempts fail to close the gap, and Stage 3 returns the best-effort selection with an explicit caveat naming the covered and unaddressed attributes.

Stage 4 finalizes the decision — a microservices architecture using a message queue, an event-sourcing database, and serverless functions — and scores it per attribute (Table IV), flagging a concrete weakness for each (e.g., for *security*: serverless functions carry function-timeout limits and reduced control over the execution environment). The two uncovered attributes’ structural components are correctly 0.0, dragging their combined scores down even where the qualitative pass alone would have scored the prose favorably — exactly the blending Section III-E argues for.

IV. EVALUATION

A. Corpus and Experimental Setup

We evaluate against the “Context Matters” replication package [3] (Zenodo, DOI: 10.5281/zenodo.18370195), derived from the same mining study that provides Stage 1’s retrieval corpus [6]. The package’s own `dataset_index.json` catalogs 954 repository folders with an extraction-confidence status each (750 Verified, 96 Doubt (name sequence), 40 Repo Inaccessible, 33 Doubt (no repo dir), 30 Doubt (missing file), 5 Doubt (file contents)); after parsing, the 882 folders with at least one accessible Markdown file yield 6,173 individual ADR files (median 4, mean 7.00, maximum 129 per repository, confirming the corpus’s documented per-repository sparsity) — the remaining 72 catalogued folders have no accessible file and contribute nothing to either count, drawn almost, but not entirely, along status lines (39 of 40 Repo Inaccessible, 32 of 33 Doubt (no repo dir), 1 of 5 Doubt (file contents); every folder under the remaining two statuses contributes at least one file). We restrict our held-out evaluation set to the 750 folders marked Verified, so ground-truth quality is not itself a confound. Following this corpus’s own upstream precedent (its companion sub-study is titled “ADR Generation from Titles”), we construct each held-out query as the record’s title and treat its full body as ground truth, matching the task formulation the retrieval-only baseline is drawn from.

The retrieval index embeds all 6,173 records with a sentence-embedding model (`all-MiniLM-L6-v2`) into a cosine-similarity nearest-neighbor index; a held-out query record is always excluded from its own retrieval results. The local, reproducible LLM backbone used for deliberation, repair, and critique throughout is Qwen2.5-1.5B-Instruct, run via `direct AutoModelForCausalLM.generate()`

calls under CUDA. We disclose deliberation’s run-time parameters because they materially affect how Table V and Table VI should be read: the pilot run used $k = 2$ and `max_rounds=1` (every agent’s propose round executed but no agent critiqued another’s candidate); the scaled run used $k = 3$ and `max_rounds=2` (one full critique round). Both runs used `max_repair_iterations=2`, this work’s full design default. Table V’s `multiagent_no_solver` and `cadence_full` columns therefore reflect propose-only deliberation, not adversarial debate; Table VI does not have this limitation.

B. Baselines and Ablation

We compare CADENCE (`cadence_full`) against four systems forming an ablation ladder that isolates retrieval’s, deliberation’s, solver verification’s, and self-critique’s contributions individually — each step adds exactly one stage over the previous one:

- **Zero-shot** (`zero_shot`): a single LLM call given only the decision context, with no retrieval, deliberation, or verification — the Dhar-et-al.-style baseline [1].
- **Retrieval-only** (`retrieval_only`): the decision context plus Stage 1’s top- k retrieved precedent titles, with no deliberation or verification — the Context-Matters-style baseline [2].
- **Multi-agent, no solver** (`multiagent_no_solver`): Stages 1–2 in full (retrieval and quality-attribute deliberation), but without Stage 3’s solver verification or Stage 4’s critique — the system most structurally comparable to MAAD’s [4] multi-agent premise without its functional-role/LLM-judge design.
- **CADENCE, no critique** (`cadence_no_critique`): Stages 1–3 (adds Stage 3’s solver-verified feasibility and repair loop over `multiagent_no_solver`), but without Stage 4’s self-critique finalization — isolates solver verification’s marginal contribution, and, compared against `cadence_full`, self-critique’s.

Every retrieval-using system routes through the same leakage guard, so no system ever retrieves or is scored against its own query record.

C. Metrics

We report the four standard generation-quality metrics used in the corpus’s own upstream evaluation work, for direct comparability: BERTScore F1, sentence-level BLEU (0–100 scale, via `sacrebleu`), ROUGE-1 F1, and METEOR, each computed against the held-out record’s real, human-authored body text. In addition, only reportable by a solver-verified system, we report *constraint-satisfaction rate* (the fraction of held-out items where Stage 3 reached `is_feasible=True`) and *average repair iterations*.

D. Results

Table V reports a small- N pilot run used to verify the full pipeline and evaluation harness end-to-end before committing to a larger run; we retain it here as a sanity-checked, fully

real data point. Table VI reports our scaled evaluation run: a separate, independent held-out sample ($N = 3$, disjoint from the pilot run above) scored under two `tactic_budget` conditions — an achievable budget ($B = 5$, matching the number of required quality attributes) and a second, tighter one ($B = 2$, included as a construction check on Stage 3’s infeasibility handling rather than as a second independent test of achievable feasibility — see the discussion immediately below) — with the three budget-independent baselines run once and shared across both conditions (they do not take a `tactic_budget` argument). Real end-to-end wall-clock cost per local-model call constrained this scaled run to $N = 3$ rather than a larger sample within the time available for this submission; both runs used the same reproducible local backbone described above, and we report the result below as obtained, rather than a larger or cherry-picked sample we did not have time to run.

Of the three budget conditions reported across both tables, only the scaled run’s $B = 5$ condition is a genuine empirical test of feasibility. With $|Q| = 5$ required quality attributes and a catalog in which every tactic supports exactly one attribute, covering all five requires at least five distinct tactics, so both the pilot’s $B = 4$ and the scaled run’s $B = 2$ are mathematically guaranteed infeasible by construction, independent of what deliberation produces; their 0% constraint-satisfaction rates are a correctness check on Stage 3’s encoding, not empirical evidence about feasibility. $B = 5$ is the one condition where feasibility is at least arithmetically possible (five tactics for five attributes, no slack), and it is where the result is informative: we expected this “achievable” budget to let `cadence_full` reach `is_feasible=True` in practice. It did not, in this $N = 3$ sample. The reason is a real, explicable property of the pipeline, not a bug: Stage 3’s tactic-extraction step (Section III-D) can only select a tactic the deliberated candidate’s terse text actually *names*, and reaching $B = 5$ feasibility requires the candidate to name five distinct tactics collectively covering all five attributes — budget headroom alone does not guarantee this if Stage 2’s synthesis converges on a candidate naming fewer, or overlapping, tactics. This run uses `max_repair_iterations=2`, this work’s full design default, superseding an earlier version that used `max_repair_iterations=1` to bound wall-clock cost. Giving the repair loop its complete budget did not change the outcome: `cadence_full` still fails to reach feasibility at $B = 5$, and `average_repair_iterations=2.00` confirms both allowed attempts were exhausted every time. This rules out an insufficient repair budget as the explanation and narrows the diagnosis to Stage 2’s tactic-naming behavior: deliberation’s terse synthesis simply does not name enough distinct, budget-relevant tactics for repair to work with, regardless of how many attempts it is given — a dependency the pilot run’s single, tighter budget condition could not have distinguished.

This run also adds `cadence_no_critique`, isolating self-critique’s marginal contribution for the first time. At $B = 5$, `cadence_full` scores higher than `cadence_no_critique` on all four generation-quality metrics, but by a systematically larger margin on the three

TABLE V

PILOT VERIFICATION RUN ($N = 3$, `TACTIC_BUDGET=4`, ALL 5 QUALITY ATTRIBUTES REQUIRED — $B = 4$ IS MATHEMATICALLY INFEASIBLE BY CONSTRUCTION UNDER THIS CATALOG, SO `CADENCE_FULL`'S 0% CONSTRAINT-SATISFACTION RATE HERE IS A CONSTRUCTION CHECK, NOT A TEST OF ACHIEVABLE FEASIBILITY; SEE SECTION IV)

System	BERTScore F1	BLEU	ROUGE-1 F1	METEOR
<code>zero_shot</code>	0.810	1.05	0.220	0.244
<code>retrieval_only</code>	0.819	1.71	0.216	0.250
<code>multiagent_no_solver</code>	0.805	1.57	0.143	0.111
<code>cadence_full</code>	0.801	0.59	0.138	0.102

`cadence_full`: constraint-satisfaction rate 0.00, average repair iterations 2.00 (see discussion below).

TABLE VI

SCALED EVALUATION RUN ($N = 3$, TWO `TACTIC_BUDGET` CONDITIONS, FULL 5-SYSTEM ABLATION LADDER — ONLY $B = 5$ IS ARITHMETICALLY ACHIEVABLE UNDER THIS CATALOG; $B = 2$ IS MATHEMATICALLY INFEASIBLE BY CONSTRUCTION AND REPORTED AS A CONSTRUCTION CHECK ON STAGE 3'S INFEASIBILITY HANDLING, NOT A SECOND FEASIBILITY TEST; SEE SECTION IV)

System	B	BERTScore F1	BLEU	ROUGE-1 F1	METEOR
<code>zero_shot</code>	—	0.824	4.24	0.300	0.252
<code>retrieval_only</code>	—	0.813	3.76	0.312	0.238
<code>multiagent_no_solver</code>	—	0.812	0.43	0.179	0.091
<code>cadence_no_critique</code>	5	0.797	0.44	0.174	0.092
<code>cadence_full</code>	5	0.806	1.04	0.198	0.100
<code>cadence_no_critique</code>	2	0.805	0.43	0.174	0.087
<code>cadence_full</code>	2	0.802	1.57	0.186	0.104

`cadence_no_critique` and `cadence_full` at both $B = 5$ and $B = 2$: constraint-satisfaction rate 0.00, average repair iterations 2.00 (the full `max_repair_iterations=2` design default — see discussion below for why this run supersedes an earlier one that used `max_repair_iterations=1`).

surface n-gram metrics (BLEU 1.04 vs. 0.44, ROUGE-1 0.198 vs. 0.174, METEOR 0.100 vs. 0.092) than on BERTScore (0.806 vs. 0.797, a 0.009 gap comparable in order of magnitude to the ~ 0.018 BERTScore spread we separately observe in the pilot run, Table VIII, and so not distinguishable from noise on its own); at $B = 2$, it scores higher on the same three n-gram metrics, with BERTScore essentially tied in the opposite direction (0.802 vs. 0.805). We therefore read the positive signal below as resting on BLEU, ROUGE-1, and METEOR's consistent agreement across both budgets, not on BERTScore. This is the first ablation comparison in this evaluation where the fuller pipeline wins consistently on the n-gram metrics rather than being roughly flat or mixed — a genuine, if modest and small-sample, positive signal that Stage 4's self-critique finalization produces text scoring better against the held-out reference than the pre-critique candidate. Both systems show the same 0% constraint-satisfaction rate at both budgets, as expected: feasibility is decided entirely within Stage 3, before Stage 4 ever runs.

Table V and Table VI draw on two genuinely disjoint held-out samples (distinct random seeds, verified to share no items), not the same three items scored twice. Across both tables, BERTScore F1 stays in a narrow 0.797–0.824 band while ROUGE-1 and METEOR are consistently, noticeably lower for the three deliberation-based systems (`multiagent_no_solver`, `cadence_no_critique`, `cadence_full`) than for the two single-pass systems. That this pattern reproduces on an independently-sampled set of held-out items is real evidence toward it being systematic rather than a small-sample artifact of either run alone — though with $N = 3$ per sample, six items total, we stop short of calling this statistically established (Section V). The constraint-satisfaction result is evidentially weaker: only $B = 5$ is a genuine test of feasibility, so its 0% result is a single

data point, not yet corroborated by an independent second run at an achievable budget.

E. Comparison Against Published Frontier-Model Results

The evaluation above compares CADENCE only against our own baseline re-implementations. To add a genuine external reference point, we identify our three scaled-run held-out items (matched by title, unique per repository) inside the “Context Matters” replication package's [3] own `Generated_ADRs` archive, which records real generations from four frontier models — Gemini-2.5-Pro [28] (proprietary; parameter count undisclosed), GLM-4.6 [31] (~ 357 B total parameters, Mixture-of-Experts; per-token active-parameter count not officially disclosed for this release), Qwen3-235B [29] (235B total, 22B active, Mixture-of-Experts), and Gemma3-4B [30] (4B, dense) — under its own `Baseline` (title-only) and `RAG_Based` ($k = 3$) generation strategies, against the same held-out ground truth we use. We deliberately do *not* use the package's own precomputed scores: a close reading of its evaluation script found it computes BLEU and METEOR with the prediction/reference roles reversed relative to standard usage and to our own code, and computes ROUGE-1 without a stemmer where we use one, making its raw numbers incomparable to ours. Instead, `scripts/extract_published_baseline_comparison.py` (in the released artifact) recomputes all four metrics from the package's raw generation/ground-truth text pairs using this paper's own `compute_corpus_metrics` — the identical BERTScore checkpoint (`roberta-large`, English, no baseline rescaling), sacrebleu-based BLEU, stemmed ROUGE-1, and METEOR computation on both sides of Table VII — so every number is directly comparable. (GLM-4.6 has no dedicated technical report at the time of writing; we cite its predecessor

TABLE VII

PUBLISHED FRONTIER-MODEL RESULTS (“CONTEXT MATTERS” PACKAGE) ON THE IDENTICAL $N = 3$ HELD-OUT ITEMS, VS. THIS WORK’S OWN SYSTEMS — ALL SCORES RECOMPUTED BY THIS PAPER’S OWN METRIC CODE FROM RAW GENERATION/REFERENCE TEXT ON BOTH SIDES (NOT THE PACKAGE’S OWN PRECOMPUTED SCORES; SEE TEXT)

System	Strategy (Size)	BERTScore F1	BLEU	ROUGE-1 F1	METEOR
<code>zero_shot</code> (this work)	— (1.5B)	0.824	4.24	0.300	0.252
Gemini-2.5-Pro	Baseline (undisclosed)	0.826	3.27	0.251	0.292
Gemma3-4B	Baseline (4B)	0.822	3.51	0.277	0.249
GLM-4.6	Baseline (~357B MoE)	0.828	5.59	0.291	0.268
Qwen3-235B	Baseline (235B/22B-active MoE)	0.837	6.84	0.317	0.285
<code>retrieval_only</code> (this work)	— (1.5B)	0.813	3.76	0.312	0.238
Gemini-2.5-Pro	RAG $k=3$ (undisclosed)	0.846	11.59	0.399	0.354
Gemma3-4B	RAG $k=3$ (4B)	0.841	11.66	0.338	0.318
GLM-4.6	RAG $k=3$ (~357B MoE)	0.856	12.24	0.381	0.357
Qwen3-235B	RAG $k=3$ (235B/22B-active MoE)	0.846	9.05	0.343	0.327
<code>cadence_full</code> ($B=5$, this work)	— (1.5B)	0.806	1.04	0.198	0.100

Size: parameter counts as publicly reported by each model’s developer (Section IV-E); “undisclosed” means the developer has not published a parameter count. BERTScore: `roberta-large` checkpoint, no baseline rescaling, identical on both sides. `cadence_full` scores below every published row on every metric; see discussion above for why we do not read this as a decision-quality gap on its own.

GLM-4.5 [31], whose architecture GLM-4.6 is a documented incremental update over.)

Two things are worth stating plainly under this harmonized recomputation. First, this is a genuine external reference point our own re-implementations cannot provide: real reported generations from models spanning roughly three orders of magnitude in scale relative to our backbone (Qwen2.5-1.5B-Instruct), not our estimate of how such models would perform. Second, the harmonized numbers give a more differentiated picture than an unharmonized comparison would suggest. Our naive baselines hold up reasonably well against the published models under the comparable strategy: `zero_shot` (BERTScore 0.824, ROUGE-1 0.300, METEOR 0.252) sits inside the published Baseline-strategy range on BERTScore and METEOR and above all four published rows on ROUGE-1, despite Gemma3-4B alone being roughly $2.7\times$ larger — a sanity check that our baselines are not artificially depressed by backbone weakness. `cadence_full`, however, is not: at $B = 5$ it scores below all eight published rows on all four metrics, including BERTScore (0.806 versus a published range of 0.822–0.856). We do not read this as evidence CADENCE is a worse decision-making *process*: the terseness asymmetry (Section V) plausibly explains part of the BLEU/ROUGE-1/METEOR gap, since CADENCE’s output is deliberately shorter than our open-ended baselines or the package’s elaborate generations. But this gap also appears on BERTScore, which we argue is length-insensitive, so length alone is not a full explanation; we consider the far larger scale of the published models (up to two orders of magnitude larger) the more likely dominant factor, though $N = 3$ is too small to disentangle scale from architecture from prompting elaborateness here. A substantially larger held-out sample and a stronger local backbone are needed before any claim about CADENCE’s absolute competitiveness against frontier-scale systems can be supported.

V. DISCUSSION

A. The BERTScore-versus-Surface-Overlap Asymmetry

Stage 2’s synthesis prompt deliberately asks for a terse, one-or-two-sentence decision plus a one-paragraph rationale, so

TABLE VIII
METRIC SPREAD ACROSS THE FOUR PILOT-RUN SYSTEMS (TABLE V),
LOWEST TO HIGHEST SCORE

Metric	Min	Max	Spread
BERTScore F1	0.801	0.819	0.018
BLEU [†]	0.59	1.71	1.12
ROUGE-1 F1	0.138	0.220	0.082
METEOR	0.102	0.244	0.142

[†]BLEU is reported on a 0–100 scale (versus 0–1 for the other three metrics), so its absolute spread is not directly comparable in raw units to the others; proportionally (spread relative to the metric’s own scale), it is comparable to ROUGE-1 and METEOR, not to BERTScore.

Stages 3 and 4 can parse it reliably — not because a terse decision is better, but because a downstream constraint solver cannot reason about open-ended prose. The `zero_shot` and `retrieval_only` baselines, by contrast, are prompted for open-ended “write an ADR” output, closer in length and register to the real, often substantially longer, reference ADR bodies that ROUGE-1 and METEOR reward via n-gram overlap. This length/register mismatch stems from our own prompt design, not decision quality, and Table V’s numbers show it plainly: BERTScore, computed over contextual embeddings rather than surface overlap, remains within 0.02 across all four systems, while ROUGE-1 and METEOR diverge by as much as 0.08 and 0.14 between the shortest- and longest-output systems. Table VIII makes this explicit, reducing Table V to the spread between each metric’s lowest- and highest-scoring system: BERTScore’s spread is an order of magnitude smaller. We consider BERTScore the fairer metric here, and flag this rather than let a reader infer a quality gap from ROUGE/METEOR that the underlying decisions do not exhibit.

We believe this asymmetry is not specific to CADENCE: any structured, multi-stage LLM pipeline whose intermediate representation is deliberately concise — for parseability, for solver-compatibility, or for any reason other than matching a free-text reference’s length — will be penalized by surface n-gram metrics relative to a single-pass system prompted for open-ended prose, independent of whether the structured pipeline’s underlying decision is better, worse, or equivalent. Evaluations of future solver-verified or otherwise structured-output LLM pipelines should report a semantic-similarity metric alongside, not instead of, surface overlap metrics for

exactly this reason.

B. Threats to Validity

Most importantly, both evaluation runs reported in Section IV are small- N ($N = 3$ each): real end-to-end wall-clock cost per local-model call, not a design choice, was the binding constraint for this submission. We report exactly what these small samples show, including the negative constraint-satisfaction result at the one budget condition ($B = 5$) where feasibility was arithmetically possible, rather than withholding it or generalizing beyond what $N = 3$ can support; giving the repair loop its full design budget did not change this result, so a larger held-out sample targeting Stage 2’s tactic-naming behavior is this work’s most direct extension. Our held-out set is also drawn from a single corpus (open-source ADR adoption); results may not transfer to proprietary practice, where decision drivers and documentation discipline differ substantially. The local LLM backbone (Qwen2.5-1.5B-Instruct) is small by current standards, chosen for reproducibility over peak capability; stronger backbones would likely shift the absolute numbers, though whether they would also change the metric-asymmetry pattern is an open question this small sample cannot settle. Our tactic catalog (26 tactics across 5 quality attributes) is deliberately small and each tactic supports exactly one attribute, setting a hard ceiling on achievable coverage under a tight budget by construction; a larger or multiply-supporting catalog would relax this and merits future work. Finally, our qualitative critique score (Stage 4) is itself LLM-generated; we take seriously the documented finding that LLMs asked to self-correct without external grounding often fail to improve, or even degrade, an already-correct answer [23], which is precisely why Stage 4’s qualitative component is blended with a deterministic structural component from Stage 3’s already solver-verified output rather than trusted alone. Even so, this blending is a mitigation, not proof of reliability, and not a substitute for human expert judgment, which we did not collect here.

C. Engineering Lessons for Solver-Verified LLM Pipelines

Two implementation-level findings generalize beyond this specific system. First, the lexicographic-versus-weighted-sum distinction in Stage 3’s optimizer (Section III-D) is not a minor tuning detail: any multi-objective encoding letting a caller supply unbounded weights for a secondary objective must either bound those weights or use a priority mechanism immune to being outvoted by scale, or it risks silently failing to enforce what the caller believed was a hard requirement. Second, the retry-on-parse-failure pattern (Section III-F) generalizes to any pipeline stage where a deterministic downstream component consumes LLM-authored structured output: parser tolerance alone does not converge, since real models produce genuinely novel wording deviations no amount of a priori regex generalization anticipates, while a bounded retry of the identical prompt exploits generation’s inherent stochasticity to route around such deviations without enumerating them in advance.

VI. CONCLUSION

We presented CADENCE, a four-stage algorithm for LLM-assisted architectural decision-making combining retrieval-augmented case-based reasoning, knowledge-graph-grounded adversarial multi-agent deliberation across ISO/IEC 25010-derived quality attributes, constraint-solver-verified feasibility with an LLM repair loop targeting the solver’s uncovered-attribute signal, and a separately-scored self-critique finalization stage. Unlike prior work, CADENCE’s deliberation represents competing quality-attribute concerns adversarially rather than as sequential functional roles, and its feasibility check is a formal solver rather than a second LLM opinion — differences that matter conceptually for any system informing decisions with real operational consequences. Our $N = 3$ ablations give a mixed but informative picture: `multiagent_no_solver` against `cadence_full` shows no measurable difference in generation-quality metrics, consistent with the metric asymmetry discussed above but also a real limit on how strongly this evaluation can support the claim; `cadence_no_critique` against `cadence_full`, by contrast, shows the fuller pipeline scoring higher consistently (all four metrics at $B = 5$, three of four at $B = 2$) — the first ablation where adding a stage helps rather than being flat, a modest positive signal for Stage 4 specifically. We implemented the complete pipeline against a real corpus of 6,173 mined ADRs, evaluated it against four baselines isolating retrieval’s, deliberation’s, solver verification’s, and self-critique’s contributions individually, and reported both standard generation-quality metrics and two metrics only a solver-verified system can report — finding constraint satisfaction to be a diagnosed negative result at this sample size (0% at the one arithmetically achievable budget tested, unchanged even with the repair loop’s full design budget) rather than a validated success, reported plainly rather than omitted. We further surfaced and explained a BERTScore-versus-surface-overlap metric asymmetry we believe applies generally to structured, multi-stage LLM pipelines, and reported two implementation-level findings — lexicographic multi-objective optimization under unbounded caller-supplied weights, and bounded retry as the general fix for unenumerable LLM output-format deviations — that generalize to solver-verified LLM systems beyond this one. Beyond our own re-implemented baselines, we located our exact held-out items within a public replication package’s real, published generations from four frontier models and compared directly against them under harmonized metric code (Section IV-E): our naive baselines held up reasonably well, but `cadence_full` scored below every published model on every metric, a gap we attribute more plausibly to the roughly two-orders-of-magnitude scale difference between our 1.5B-parameter backbone and these systems than to CADENCE’s algorithmic design, though our small sample cannot fully separate the two. Future work includes a substantially larger held-out sample to determine whether CADENCE can reach solver-verified feasibility at an achievable budget — narrowed by this study specifically to Stage 2’s tactic-naming behavior — alongside a tactic catalog permitting multi-attribute coverage per tactic, stronger LLM

backbones, and human expert judgments of decision quality to complement the automatic and solver-verified metrics reported here.

DATA AND CODE AVAILABILITY

The complete CADENCE implementation — retrieval index, 26-tactic knowledge graph, every deliberation/solver/critique prompt, evaluation harness, scripts/run_worked_example.py with its committed output underlying Table IV, scripts/extract_published_baseline_comparison.py with its recomputed results underlying Table VII, and the scaled run's aggregate per-system results underlying Table VI (per-system, not yet per-item) — is publicly available at:

<https://github.com/vinhq dang/Large-Language-Models-in-Service-Oriented-Ecosystems-Design>

The pilot run (Table V) predates this artifact's current results layout; its numbers are reported in the table itself, but no separate per-run results file is currently committed for it.

ACKNOWLEDGMENTS

The author thanks the maintainers of the “Context Matters” ADR replication package for making a real, richly annotated ADR corpus publicly available, without which this work's retrieval stage and evaluation would not have been possible.

REFERENCES

- [1] R. Dhar, K. Vaidhyanathan, and V. Varma, “Can LLMs Generate Architectural Design Decisions? – An Exploratory Empirical Study,” in *Proc. IEEE 21st Int. Conf. Software Architecture (ICSA)*, 2024. arXiv:2403.01709.
- [2] A. Gupta, R. Dhar, D. Feitosa, and K. Vaidhyanathan, “Context Matters: Evaluating Context Strategies for Automated ADR Generation Using LLMs,” accepted at *EASE 2026 Research Track*. arXiv:2604.03826.
- [3] A. Gupta, R. Dhar, D. Feitosa, and K. Vaidhyanathan, “Context Matters – Replication Package,” Zenodo, 2026, doi: 10.5281/zenodo.18370195.
- [4] R. Li, Y. Zhang, X. Zhou, P. Liang, W. Sun, J. Xuan, Z. Jin, and Y. Liu, “MAAD: Automate Software Architecture Design through Knowledge-Driven Multi-Agent Collaboration,” arXiv preprint, 2025. arXiv:2507.21382.
- [5] X. Zhou, R. Li, P. Liang, B. Zhang, M. Shahin, Z. Li, and C. Yang, “Using LLMs in Generating Design Rationale for Software Architecture Decisions,” accepted for publication in *ACM Trans. Softw. Eng. Methodol. (TOSEM)*, 2025. arXiv:2504.20781.
- [6] G. Buchgeher, R. Schöberl, P. Geist, J. Dorninger, C. Haindl, and R. Weinreich, “Using Architecture Decision Records in Open Source Projects – An MSR Study on GitHub,” *IEEE Access*, vol. 11, pp. 63725–63740, 2023, doi: 10.1109/ACCESS.2023.3287654.
- [7] R. Dhar, A. Kakran, A. Karan, K. Vaidhyanathan, and V. Varma, “DRAFT-ing Architectural Design Decisions using LLMs,” arXiv preprint, 2025. arXiv:2504.08207.
- [8] R. Su, A. Bakhtin, N. Ahmad, M. Esposito, V. Lenarduzzi, and D. Taibi, “Evaluating Large Language Models for Detecting Architectural Decision Violations,” arXiv preprint, 2026. arXiv:2602.07609.
- [9] ISO/IEC, *ISO/IEC 25010:2023 – Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuARE) – Product Quality Model*, 2nd ed. Geneva, Switzerland: Int. Organization for Standardization, 2023.
- [10] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed. Boston, MA, USA: Addison-Wesley Professional, 2021, ISBN: 978-0-13-688609-9.
- [11] I. Kumara, H. Kaplan, J. Owotogbe, D. A. Tamburri, and W.-J. van den Heuvel, “Large Language Models for Service-Oriented Computing (LLM4SOC): Review and Research Directions,” in *Proc. SummerSOC 2025*, Communications in Computer and Information Science, vol. 2602, Springer, Cham, 2026, doi: 10.1007/978-3-032-07313-6_6.
- [12] K. Sellami and M. A. Saied, “Contrastive Learning-Enhanced Large Language Models for Monolith-to-Microservice Decomposition,” *Empirical Softw. Eng.*, 2025, doi: 10.1007/s10664-025-10732-z. arXiv:2502.04604.
- [13] D. Albuquerque, J. Renan, G. Rodríguez, E. Dantas, A. França, M. Perkusich, K. Gorgônio, and A. Perkusich, “From Textual Requirements to Microservice Architectures: A Comprehensive Evaluation of LLM-Based Design Synthesis,” arXiv preprint, 2026. arXiv:2607.28307.
- [14] L. Pan, A. Albalak, X. Wang, and W. Y. Wang, “Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning,” in *Findings of the Assoc. Comput. Linguistics: EMNLP 2023*, pp. 3806–3824, doi: 10.18653/v1/2023.findings-emnlp.248.
- [15] T. X. Olausson, A. Gu, B. Lipkin, C. E. Zhang, A. Solar-Lezama, J. B. Tenenbaum, and R. Levy, “LINC: A Neurosymbolic Approach for Logical Reasoning by Combining Language Models with First-Order Logic Provers,” in *Proc. 2023 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, pp. 5153–5176, doi: 10.18653/v1/2023.emnlp-main.313. arXiv:2310.15164.
- [16] S. Szeider, “Bridging Language Models and Symbolic Solvers via the Model Context Protocol,” in *Proc. 28th Int. Conf. Theory and Applications of Satisfiability Testing (SAT 2025)*, LIPIcs vol. 341, doi: 10.4230/LIPIcs.SAT.2025.30.
- [17] Y. Hao, Y. Chen, Y. Zhang, and C. Fan, “Large Language Models Can Solve Real-World Planning Rigorously with Formal Verification Tools,” arXiv preprint, 2024. arXiv:2404.11891.
- [18] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, “Improving Factuality and Reasoning in Language Models through Multiagent Debate,” in *Proc. 41st Int. Conf. Machine Learning (ICML)*, PMLR vol. 235, pp. 11733–11763, 2024. arXiv:2305.14325.
- [19] T. Liang, Z. He, W. Jiao, X. Wang, Y. Wang, R. Wang, Y. Yang, S. Shi, and Z. Tu, “Encouraging Divergent Thinking in Large Language Models through Multi-Agent Debate,” in *Proc. 2024 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2024. arXiv:2305.19118.
- [20] C.-M. Chan, W. Chen, Y. Su, J. Yu, W. Xue, S. Zhang, J. Fu, and Z. Liu, “ChatEval: Towards Better LLM-based Evaluators through Multi-Agent Debate,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024. arXiv:2308.07201.
- [21] A. Madaan *et al.*, “Self-Refine: Iterative Refinement with Self-Feedback,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.17651.
- [22] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language Agents with Verbal Reinforcement Learning,” in *Proc. NeurIPS*, 2023. arXiv:2303.11366.
- [23] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, “Large Language Models Cannot Self-Correct Reasoning Yet,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024. arXiv:2310.01798.
- [24] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Proc. NeurIPS*, 2020. arXiv:2005.11401.
- [25] Y. Tao, Y. Li, Y. Qin, and Y. Liu, “Retrieval-Augmented Code Generation: A Survey with Focus on Repository-Level Approaches,” arXiv preprint, 2025. arXiv:2510.04905.
- [26] N. Wiratunga, R. Abeyratne, L. Jayawardena, K. Martin, S. Massie, I. Nkisi-Orji, R. Weerasinghe, A. Liret, and B. Fleisch, “CBR-RAG: Case-Based Reasoning for Retrieval Augmented Generation in LLMs for Legal Question Answering,” in *Proc. Int. Conf. Case-Based Reasoning (ICCBR)*, 2024. arXiv:2404.04302.
- [27] K. Hatalis, D. Christou, and V. Kondapalli, “Review of Case-Based Reasoning for LLM Agents: Theoretical Foundations, Architectural Components, and Cognitive Integration,” arXiv preprint, 2025. arXiv:2504.06943.
- [28] G. Comanici, E. Bieber, M. Schaeckermann, I. Pasupat, N. Sachdeva, *et al.*, “Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities,” arXiv preprint, 2025. arXiv:2507.06261.
- [29] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, *et al.* (Qwen Team), “Qwen3 Technical Report,” arXiv preprint, 2025. arXiv:2505.09388.
- [30] Gemma Team, A. Kamath, J. Ferret, S. Pathak, N. Vieillard, *et al.*, “Gemma 3 Technical Report,” arXiv preprint, 2025. arXiv:2503.19786.
- [31] A. Zeng, X. Lv, Q. Zheng, Z. Hou, B. Chen, *et al.*, “GLM-4.5: Agentic, Reasoning, and Coding (ARC) Foundation Models,” arXiv preprint, 2025. arXiv:2508.06471.